



dq800-lab3-tsql-avanzado

Informe de Práctica — Lab: Write advanced T-SQL queries (DQ-800)

Módulo: DP-800

Práctica: Write advanced T-SQL queries (JSON, CTE, funciones de ventana, OPENJSON)

Base de datos: AdventureWorksLT

Fuente del enunciado: [Microsoft Learn — Write advanced T-SQL queries](#)

1. Introducción

En el laboratorio 3 de la DP-800 se van a generar de productos en formato JSON para un catalogo web y crear informes que rankearan productos dentro de cada categoría

1.1 Objetivos

- Generar salida JSON a partir de datos de productos con `FOR JSON PATH`.
- Crear estructuras JSON anidadas con `JSON_OBJECT`.
- Combinar salida JSON con una CTE y una función de ventana (`ROW_NUMBER()`).
- Parsear datos JSON con `OPENJSON` y unirlos con tablas existentes.

1.2 Prerrequisitos

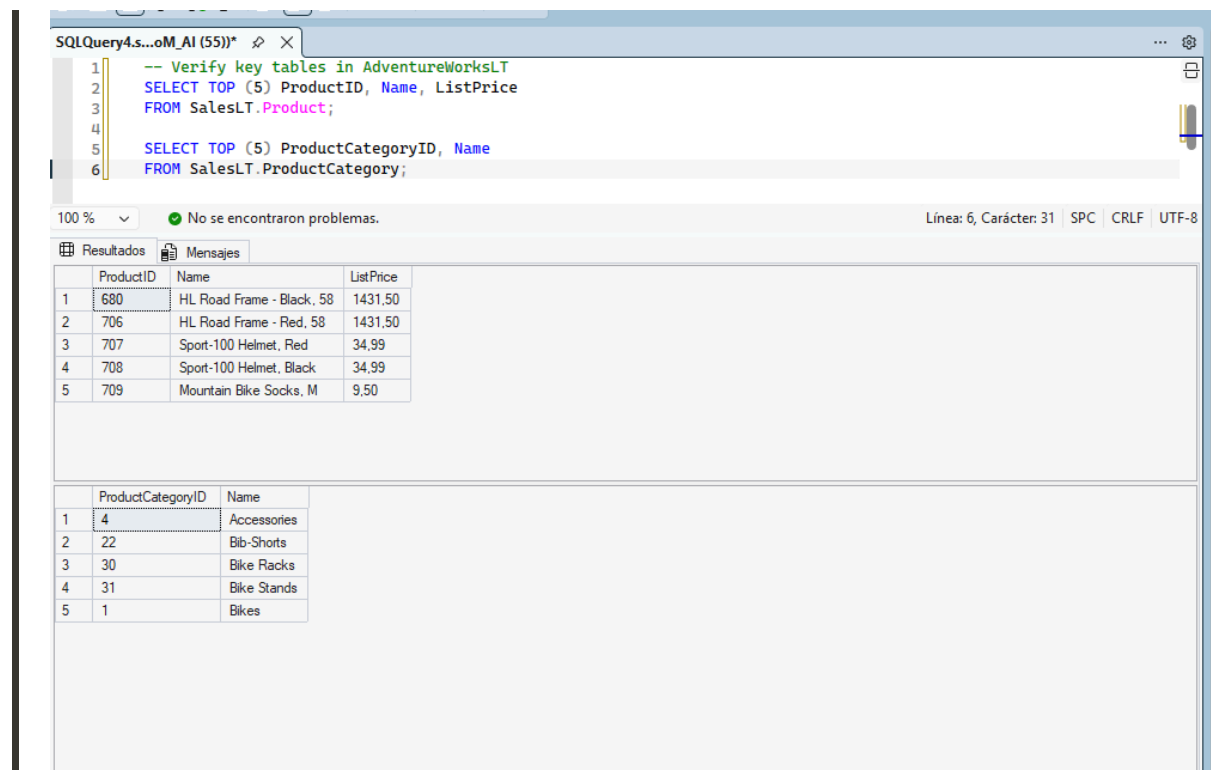
- SQL Server 2022+ o Azure SQL Database.
- Herramienta de consultas (por ejemplo, SQL Server Management Studio).
- Conexión con permisos de lectura.
- Base de datos de ejemplo [AdventureWorksLT](#).

2. Conexión a AdventureWorksLT

Verificación de la conectividad con la base de datos y de la existencia de las tablas clave.

```
-- Verify key tables in AdventureWorksLT
SELECT TOP (5) ProductID, Name, ListPrice
FROM SalesLT.Product;

SELECT TOP (5) ProductCategoryID, Name
FROM SalesLT.ProductCategory;
```



The screenshot shows a SQL Server query window with two queries executed. The first query, 'SELECT TOP (5) ProductID, Name, ListPrice FROM SalesLT.Product;', returns a table with 5 rows of product data. The second query, 'SELECT TOP (5) ProductCategoryID, Name FROM SalesLT.ProductCategory;', returns a table with 5 rows of product category data.

ProductID	Name	ListPrice
680	HL Road Frame - Black, 58	1431.50
706	HL Road Frame - Red, 58	1431.50
707	Sport-100 Helmet, Red	34.99
708	Sport-100 Helmet, Black	34.99
709	Mountain Bike Socks, M	9.50

ProductCategoryID	Name
4	Accessories
22	Bib-Shorts
30	Bike Racks
31	Bike Stands
1	Bikes

3. Construcción de salida JSON a partir de datos de productos

3.1 Crear un objeto JSON por producto

Uso de `FOR JSON PATH` para convertir filas de productos en JSON.

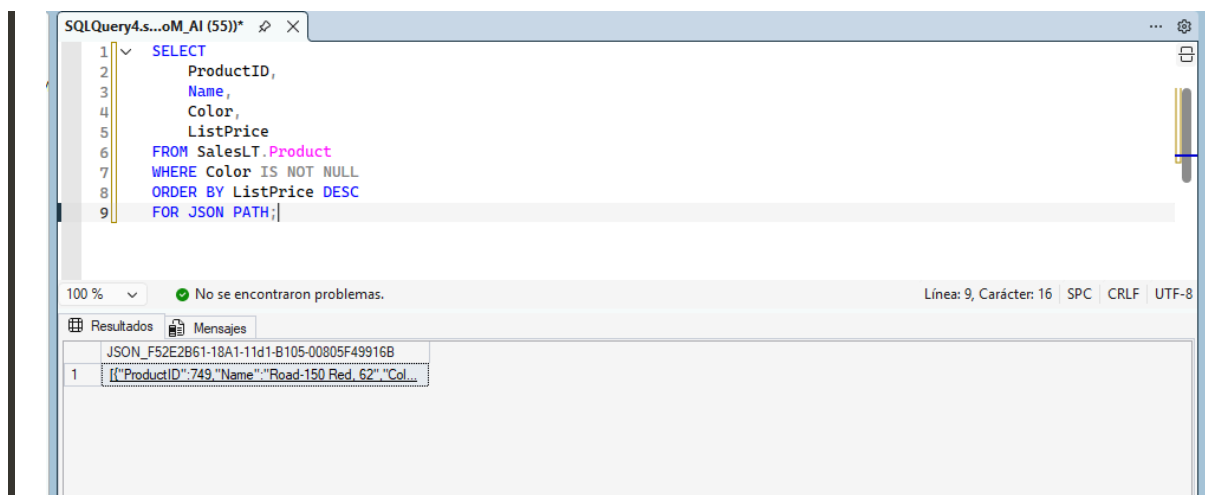
```
SELECT
    ProductID,
    Name,
    Color,
```

```

ListPrice
FROM SalesLT.Product
WHERE Color IS NOT NULL
ORDER BY ListPrice DESC
FOR JSON PATH;

```

La clausula **FOR JSON PATH** se encarga de convertir en arrays json que corresponden las filas a las columnas



3.2 Crear JSON anidado con categorías de producto

Se añade información de categoría como un objeto anidado mediante

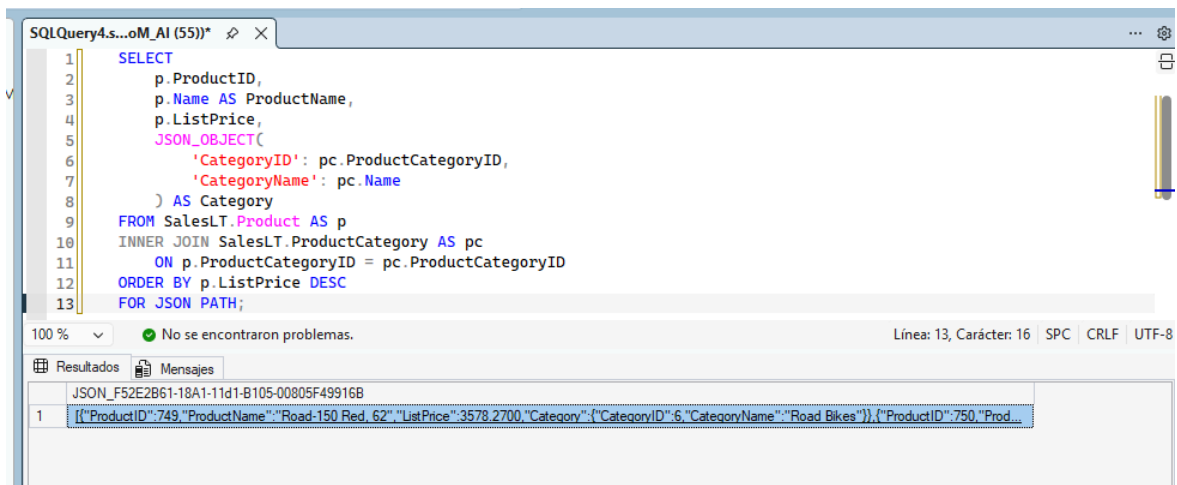
JSON_OBJECT.

```

SELECT
    p.ProductID,
    p.Name AS ProductName,
    p.ListPrice,
    JSON_OBJECT(
        'CategoryID': pc.ProductCategoryID,
        'CategoryName': pc.Name
    ) AS Category
FROM SalesLT.Product AS p
INNER JOIN SalesLT.ProductCategory AS pc
    ON p.ProductCategoryID = pc.ProductCategoryID
ORDER BY p.ListPrice DESC
FOR JSON PATH;

```

La anidación de JSON nos permite estructurar más el objeto JSON que nos devuelve la transformación de las consulta



The screenshot shows a SQL query window with the following code:

```
1 SELECT
2     p.ProductID,
3     p.Name AS ProductName,
4     p.ListPrice,
5     JSON_OBJECT(
6         'CategoryID': pc.ProductCategoryID,
7         'CategoryName': pc.Name
8     ) AS Category
9 FROM SalesLT.Product AS p
10 INNER JOIN SalesLT.ProductCategory AS pc
11     ON p.ProductCategoryID = pc.ProductCategoryID
12 ORDER BY p.ListPrice DESC
13 FOR JSON PATH;
```

Below the query, the results pane shows a single row of JSON data:

```
1 [{"ProductID":749,"ProductName":"Road-150 Red, 62","ListPrice":3578.2700,"Category":{"CategoryID":6,"CategoryName":"Road Bikes"}}, {"ProductID":750,"Prod...
```

4. Combinar JSON con una CTE y una función de ventana

4.1 CTE con ranking mediante función de ventana

Construcción de la lógica de la consulta usando una CTE y `ROW_NUMBER()`.

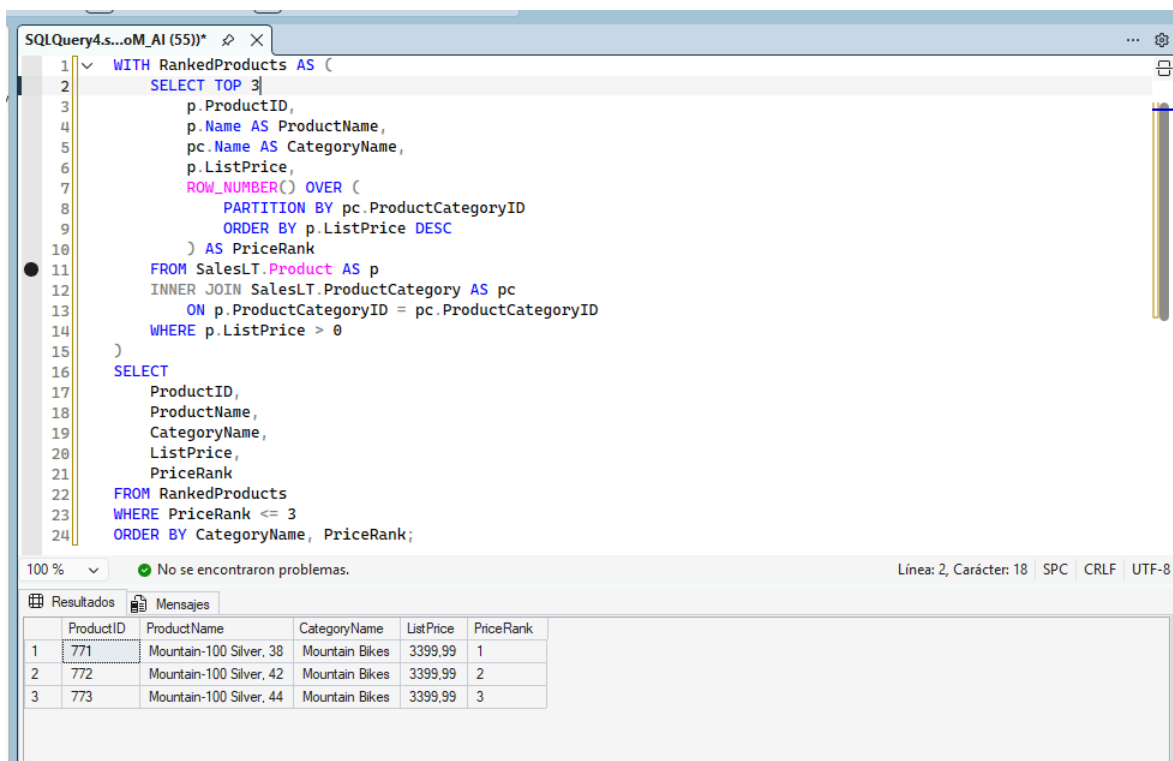
```
WITH RankedProducts AS (
    SELECT
        p.ProductID,
        p.Name AS ProductName,
        pc.Name AS CategoryName,
        p.ListPrice,
        ROW_NUMBER() OVER (
            PARTITION BY pc.ProductCategoryID
            ORDER BY p.ListPrice DESC
        ) AS PriceRank
    FROM SalesLT.Product AS p
    INNER JOIN SalesLT.ProductCategory AS pc
        ON p.ProductCategoryID = pc.ProductCategoryID
    WHERE p.ListPrice > 0
)
SELECT
    ProductID,
    ProductName,
```

```

        CategoryName,
        ListPrice,
        PriceRank
FROM RankedProducts
WHERE PriceRank <= 3
ORDER BY CategoryName, PriceRank;

```

El CTE calcula una clasificación de precios para cada producto dentro de su categoría. La cláusula reinicia la numeración de cada categoría y asigna rango 1 al producto más caro. La consulta externa filtra solo los 3 productos principales por categoría. `PARTITION BY ORDER BY ListPrice DESC`



The screenshot shows a SQL query window with the following code:

```

1  WITH RankedProducts AS (
2      SELECT TOP 3
3          p.ProductID,
4          p.Name AS ProductName,
5          pc.Name AS CategoryName,
6          p.ListPrice,
7          ROW_NUMBER() OVER (
8              PARTITION BY pc.ProductCategoryID
9              ORDER BY p.ListPrice DESC
10         ) AS PriceRank
11     FROM SalesLT.Product AS p
12     INNER JOIN SalesLT.ProductCategory AS pc
13         ON p.ProductCategoryID = pc.ProductCategoryID
14     WHERE p.ListPrice > 0
15 )
16 SELECT
17     ProductID,
18     ProductName,
19     CategoryName,
20     ListPrice,
21     PriceRank
22 FROM RankedProducts
23 WHERE PriceRank <= 3
24 ORDER BY CategoryName, PriceRank;

```

Below the query window, the results grid shows the following data:

	ProductID	ProductName	CategoryName	ListPrice	PriceRank
1	771	Mountain-100 Silver, 38	Mountain Bikes	3399.99	1
2	772	Mountain-100 Silver, 42	Mountain Bikes	3399.99	2
3	773	Mountain-100 Silver, 44	Mountain Bikes	3399.99	3

4.2 Exportar los productos rankeados como JSON

Se añade `FOR JSON PATH` con `ROOT('TopProducts')` para dar formato a los resultados como si fueran para una API.

```

WITH RankedProducts AS (
    SELECT
        p.ProductID,
        p.Name AS ProductName,
        pc.Name AS CategoryName,

```

```

        p.ListPrice,
        ROW_NUMBER() OVER (
            PARTITION BY pc.ProductCategoryID
            ORDER BY p.ListPrice DESC
        ) AS PriceRank
    FROM SalesLT.Product AS p
    INNER JOIN SalesLT.ProductCategory AS pc
        ON p.ProductCategoryID = pc.ProductCategoryID
    WHERE p.ListPrice > 0
)
SELECT
    ProductID,
    ProductName,
    CategoryName,
    ListPrice,
    PriceRank
FROM RankedProducts
WHERE PriceRank <= 3
ORDER BY CategoryName, PriceRank
FOR JSON PATH, ROOT('TopProducts');

```

The screenshot shows a SQL Server Enterprise Manager window with a query titled "SQLQuery4.s...oM_AI (55)". The query is the same as the one in the previous block. The status bar at the bottom indicates "100 %", "No se encontraron problemas.", "Línea: 25, Carácter: 36", "SPC", "CRLF", and "UTF-8".

The "Resultados" (Results) pane shows a single row of JSON data:

Resultados	
1	["TopProducts":["ProductID":855,"ProductName":"Men's Bib-Shorts, S","CategoryName":"Bib-Shorts","ListPrice":89.9900,"PriceRank":1],"Product...

5. Parsear datos JSON con OPENJSON

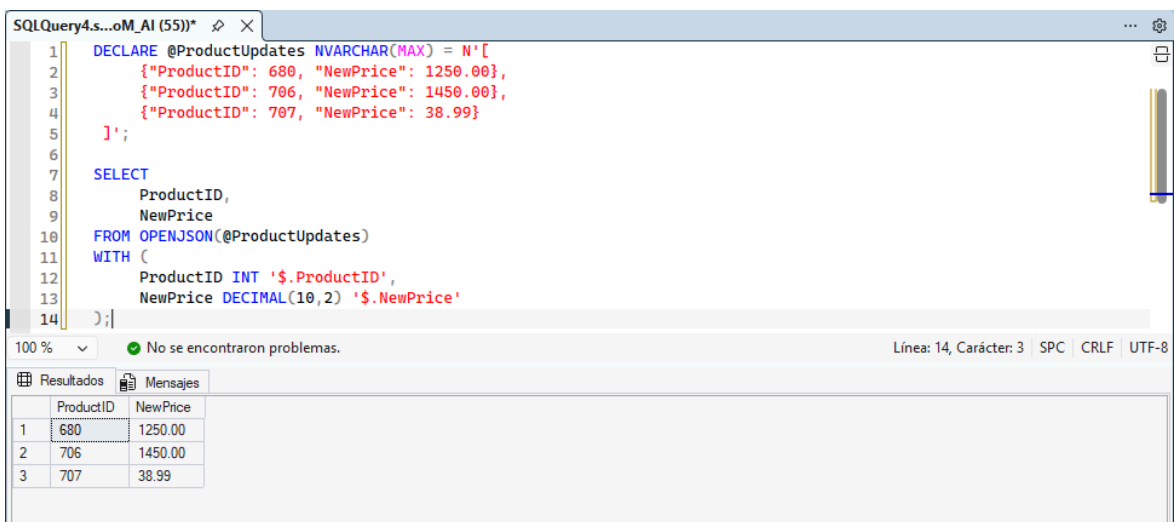
5.1 Parsear un array JSON en filas

Uso de `OPENJSON` para convertir una cadena JSON de actualizaciones de productos en una tabla.

```
DECLARE @ProductUpdates NVARCHAR(MAX) = N'[
  {"ProductID": 680, "NewPrice": 1250.00},
  {"ProductID": 706, "NewPrice": 1450.00},
  {"ProductID": 707, "NewPrice": 38.99}
]';

SELECT
    ProductID,
    NewPrice
FROM OPENJSON(@ProductUpdates)
WITH (
    ProductID INT '$.ProductID',
    NewPrice DECIMAL(10,2) '$.NewPrice'
);
```

 **[RELLENAR CON TUS PALABRAS: Explica para qué sirve la cláusula WITH en OPENJSON y qué indica la sintaxis \$.PropertyName]**



The screenshot shows a SQL query window titled "SQLQuery4.s...oM_AI (55)". The query is the same as the one in the previous block. Below the query, the results are displayed in a table with two columns: "ProductID" and "NewPrice". The results are as follows:

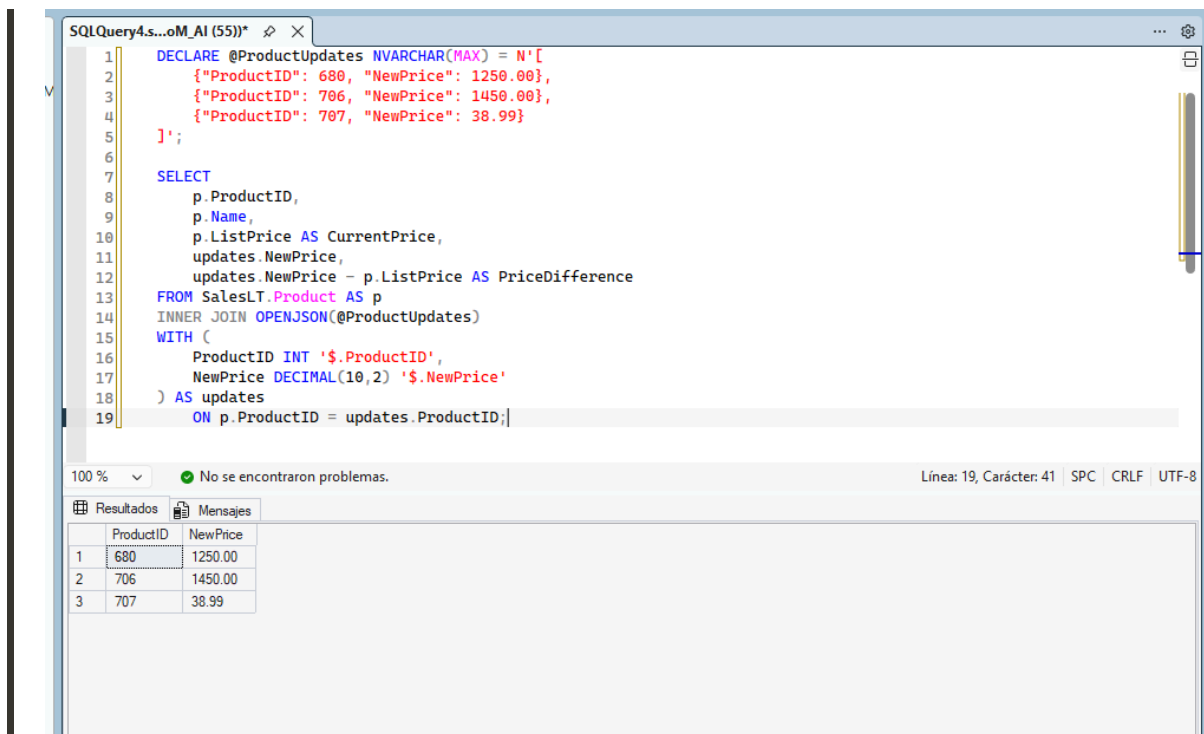
	ProductID	NewPrice
1	680	1250.00
2	706	1450.00
3	707	38.99

5.2 Unir el JSON parseado con datos existentes

Combinación de los datos JSON parseados con la tabla `Product` para comparar precio actual y precio nuevo.

```
DECLARE @ProductUpdates NVARCHAR(MAX) = N'[
  {"ProductID": 680, "NewPrice": 1250.00},
  {"ProductID": 706, "NewPrice": 1450.00},
  {"ProductID": 707, "NewPrice": 38.99}
]';

SELECT
  p.ProductID,
  p.Name,
  p.ListPrice AS CurrentPrice,
  updates.NewPrice,
  updates.NewPrice - p.ListPrice AS PriceDifference
FROM SalesLT.Product AS p
INNER JOIN OPENJSON(@ProductUpdates)
WITH (
  ProductID INT '$.ProductID',
  NewPrice DECIMAL(10,2) '$.NewPrice'
) AS updates
ON p.ProductID = updates.ProductID;
```



The screenshot shows a SQL Server Enterprise Manager window with a T-SQL query editor. The query is a stored procedure that updates product prices based on a JSON input. The query is as follows:

```
1 DECLARE @ProductUpdates NVARCHAR(MAX) = N'[
2     {"ProductID": 680, "NewPrice": 1250.00},
3     {"ProductID": 706, "NewPrice": 1450.00},
4     {"ProductID": 707, "NewPrice": 38.99}
5 ]';
6
7 SELECT
8     p.ProductID,
9     p.Name,
10    p.ListPrice AS CurrentPrice,
11    updates.NewPrice,
12    updates.NewPrice - p.ListPrice AS PriceDifference
13 FROM SalesLT.Product AS p
14 INNER JOIN OPENJSON(@ProductUpdates)
15 WITH (
16     ProductID INT '$.ProductID',
17     NewPrice DECIMAL(10,2) '$.NewPrice'
18 ) AS updates
19 ON p.ProductID = updates.ProductID;
```

Below the query editor, the status bar indicates "No se encontraron problemas." and "Línea: 19, Carácter: 41 SPC CRLF UTF-8". The results pane shows the following data:

	ProductID	NewPrice
1	680	1250.00
2	706	1450.00
3	707	38.99

6. Limpieza de recursos

No se eliminan los recursos

7. Conclusiones

Todas las funciones para trabajar con JSON muy útiles para la exportación de los datos a aplicaciones o apis que trabajen con tu base de datos